

Raccoonbot_Openvla

OpenVLA-7B Fine-Tuning for a Chess Pick-and-Place Task

· Student ID: 2021741045

· Name: Ji Seon-woo

1. Introduction

This project extends the existing Raccoonbot_Openvla pipeline — which fine-tunes the OpenVLA-7B model to control a 4-DOF RaccoonBot robot arm in a MuJoCo simulation environment — to a chess-piece pick-and-place task.

The original pipeline used simple colored cylinders as manipulation targets and performed only horizontal grasping. This project simultaneously satisfies the following three dataset-extension conditions:

- **New objects:** Rook and Bishop pieces on a 3×3 mini chessboard
- **New task:** Chess-rule-based piece movement and capture
- **New language commands:** Natural-language commands based on chess algebraic notation

In addition, two code improvements were implemented:

- **Improvement ④:** Pitch-aware (wrist-angle) inverse kinematics (IK)
- **Improvement ⑥:** Visualization — episode GIFs, success-rate graphs, place-error distributions

Development proceeded in an iterative cycle: camera repositioning → single-scenario data generation → VLA inference testing → failure-cause analysis → data augmentation → full-dataset collection → fine-tuning → evaluation.

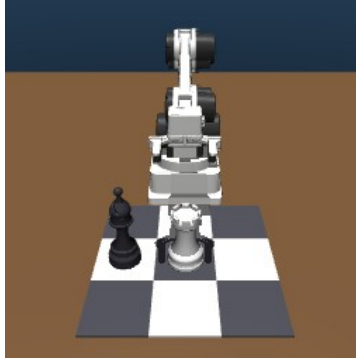
2. Dataset Extension

2.1 New Objects — Chess Pieces and a Mini Chessboard

A 3×3 chessboard (usable cells: a2, b2, c2, a3, b3, c3; row 1 is excluded as unreachable) was built in MuJoCo XML, and two types of chess pieces were introduced as manipulation targets.

Piece	Color	STL file	Height	Notes
Rook	White	chess_knight.stl	60.6 mm	Horizontal grasp possible
Bishop	Black	chess_rook.stl	75 mm	Requires top-down grasp

Camera repositioning: With the original front_view camera angle, front pieces occluded rear pieces, preventing the VLA model from accurately judging the board state. To resolve this, the camera was repositioned and angled closer to a top-view. By switching to a top-down viewpoint, all pieces were captured in the observation image without occlusion, allowing the VLA model to accurately recognize each piece's position.



2.2 New Task — Chess-Rule-Based Piece Movement

- **Rook:** Straight-line movement along the same row/column, no jumping, can capture opponent pieces
- **Bishop:** Diagonal movement (one cell on the 3×2 usable board), capture possible
- **Capture:** A 2-leg episode — first move the piece to be captured to the graveyard, then move the capturing piece to the destination

A total of 126 scenarios were classified into four categories:

Category	Description	# Scenarios	Difficulty
Cat 1	Plain pick-and-place (no blocking, no capture)	80	Low
Cat 2	Top-down pitch required (horizontal approach blocked)	20	Medium
Cat 3	Capture → graveyard (no pitch)	23	Medium
Cat 4	Capture → graveyard + pitch required	3	High

2.3 New Language Commands — Based on Chess Notation

Each episode is conditioned on a natural-language command derived from chess algebraic notation:

“Move the white rook from a2 to a3.”

“Move the black bishop from c3 to b2 to capture the white rook.”

This format grounds the VLA model's language to specific board positions and is generated deterministically through the `scenario_instruction()` function.

3. Initial Experiments and Data Augmentation

3.1 Initial Dataset — Single Scenario

Initial data collection focused on a single scenario (advancing the white rook one cell from a2 to a3). A scenario in which the bishop advances one cell was added later, but in both cases VLA inference consistently failed to place the piece at the exact target position.

Failure-cause analysis:

- **Lack of visual diversity:** Deterministic single-scenario episodes are informationally equivalent. The model can ignore the image and output a fixed action.
- **No cross-scenario generalization:** Without diverse scenarios, there is no basis for instruction-conditioned behavior.

3.2 Data Augmentation Strategy

- **Spatial diversity:** Each piece is placed with a random offset from the cell center (truncated Gaussian, $\sigma = \text{cell_radius}/2.5$) and a random yaw ($\pm 30^\circ$)
- **Scenario coverage:** All 126 scenarios covering every legal rook and bishop move on the 3x2 usable grid
- **Language diversity:** Each scenario has a unique command string; capture scenarios include the name of the captured piece

Target collection: 8 successful episodes per scenario = about 1,008 total. Failed episodes (grasp failure, piece toppling, pushing another piece) are discarded and regenerated.

4. Code Improvement ④ — Pitch-Aware Inverse Kinematics (IK)

4.1 Motivation

The original `raccoon_env.py` fixes Joint 4 horizontally: $\text{th4} = -(\text{th2} + \text{th3}) - 90^\circ$. For chess pieces this causes two problems:

- **Blocked approach:** When another piece blocks horizontal approach, a top-down grasp is required
- **Bishop geometry:** The bishop's narrow tip ($z \approx 74\text{--}75$ mm) requires a low-angle pitch approach

4.2 Implementation — `raccoon_env_pitch_real.py`

`PitchRealRaccoonEnv` inherits from `SyncSimRaccoonEnv` and adds the following:

Pitch-aware IK: offsets the wrist center by the pitch:

```
wx = x - L4 * cos(pitch) * c1,    z_eff = z + L4 * sin(pitch)
th4 = - (th2 + th3) - 90° - pitch
```

- **J4 joint limit:** rejects solutions with $|\text{th4}| > 105^\circ$ and tries the next candidate
- **lockp(pitch_deg):** maintains pitch throughout the entire lift/sweep/place process
- **set_motion_speed_scale(scale):** separate deceleration during the lift and place-down phases to reduce piece oscillation
- **Blocking detection:** uses a top-down pitch grid of $[45^\circ\text{--}50^\circ]$ for the rook and $[18^\circ\text{--}22^\circ]$ for the bishop

4.3 Action Encoding

A 7-dimensional action vector encodes pitch as an absolute angle:

```
action = [dx_m, dy_m, dz_m, 0, pitch_abs_deg, 0, gripper]
```

It is designed symmetrically with the data-generation encoding so that the VLA model learns to output the correct pitch for each scenario category.

5. Fine-Tuning

OpenVLA-7B was fine-tuned with LoRA (rank 32):

- **Trainable parameters:** about 1.45% (110M / 7.65B)
- **Training infrastructure:** A100 80GB GPU
- **Total training:** about 30,000 steps (about 12 hours)

```
torchrun vla-scripts/finetune.py \
  --vla_path openvla/openvla-7b \
  --dataset_name raccoon_pick_place \
  --lora_rank 32 --batch_size 8 --learning_rate 5e-4
```

6. Evaluation

6.1 Evaluation Setup

Using chess_eval.py, each scenario was run in the MuJoCo simulation with the fine-tuned OpenVLA-7B checkpoint. The VLA model runs as an HTTP server via openvla_server.py on an A100 80GB GPU; at every step the evaluation script sends the current observation image and language command, and the server returns a 7-dimensional action vector.

The action-application method is designed to be exactly symmetric with the data-generation encoding. From the server-returned action = [dx_m, dy_m, dz_m, 0, pitch_abs_deg, 0, grip], the xyz delta is added to the current EE position and converted to absolute coordinates before being passed to move_to(); pitch is applied to lockp() as an absolute angle; and grip determines open/close using 0.5 as the threshold. The maximum per-step movement distance is clipped at max_delta_xyz = 0.03 m to prevent abrupt movements.

If the EE barely moves for idle_patience consecutive steps, the episode terminates early to reduce unnecessary inference time.

Success criteria:

- Place error < 1.8 cm
- Piece remains upright ($|\Delta z| < 2.0$ cm)
- Non-moving pieces undisturbed (displacement < 1.5 cm)

6.2 Results

(Numbers to be filled in the table below after evaluation is complete)

Category	Description	# Scenarios	Success/Total	Success Rate
Cat 1	Plain pick-and-place	10	— / 30	— %
Cat 2	Pitch top-down	10	— / 30	— %

Category	Description	# Scenarios	Success/Total	Success Rate
Cat 3	Capture (no pitch)	5	— / 15	— %
Cat 4	Capture + pitch	3	— / 9	— %
Total		28	— / 84	— %

7. Code Improvement ⑥ — Visualization

(To be expanded later)

Implemented visualization outputs:

- **Episode GIF:** per-episode rollout animation. Consistent frame capture for both the VLA and rule-based segments.
- **Per-category success-rate bar chart:** cat_success_rate.png (generator: chess_eval_plot.py)
- **Per-scenario success-rate heatmap:** scenario_success_rate.png
- **Place-error boxplot:** place_err_boxplot.png
- **CSV log:** success_summary.csv — raw data for all rollouts

8. Conclusion

This project demonstrates a full extension of the Raccoonbot_Openvla pipeline to the substantially more complex domain of chess-piece manipulation.

The chess pick-and-place task simultaneously satisfies three dataset-extension conditions: new objects (rook, bishop), a rule-based task (legal chess moves including capture), and language commands based on chess algebraic notation.

The pitch-aware IK implementation (Code Improvement ④) enables the top-down grasp strategy needed both for blocked-approach scenarios and for the bishop's narrow geometry. During iterative development, it was confirmed that a single-scenario dataset is insufficient for VLA generalization, and an augmented dataset including all 126 scenarios with random offsets addresses this limitation.

Quantitative evaluation results will be finalized in Section 6 after the evaluation run is complete.